Network-Aware Event Sequence Modeling for User-Behavior and System Log Anomaly Detection

Thanh Tien Cao[1,2], Tu Hoang Trinh[2⋆], and Ha Manh Tran[2*]

[1] Industrial University of Ho Chi Minh City, Ho Chi Minh City, Vietnam
thanhct25471@pgr.iuh.edu.vn
[2] Faculty of Information Technology, Ho Chi Minh City University of Foreign Languages - Information Technology, Ho Chi Minh City, Vietnam
thanhct@huflit.edu.vn, tht.csec2005@gmail.com, hatm@huflit.edu.vn

Abstract. This paper presents a network-aware system log anomaly detection pipeline based on Drain3 and event sequence analysis, modeling log sequences as walks on directed event-transition networks and client-service interaction graphs. We evaluate Temporal Convolutional Networks (TCN) and Transformers against traditional baselines on three large-scale log datasets. Raw logs are parsed into event templates, grouped into sequences, and scored for anomalies using five branches: PCA/SVD, Isolation Forest, DeepLog (LSTM next-event prediction), TCN (Temporal Convolutional Network), and Transformer masked event modeling. On HDFS (500k lines, 137 templates), experiments across 5 random seeds show that PCA achieves an F1-score of 0.5711 ± 0.0944, slightly outperforming DeepLog and TCN (0.4613 ± 0.0199). On BGL (500k lines, 198 templates, window size W=100), TCN leads with an F1-score of 0.9826 ± 0.0022 (top-k mode) and 0.9474 ± 0.0035 (NLL threshold mode), outperforming DeepLog LSTM (0.9820 ± 0.0042 and 0.9381 ± 0.0075) as well as the val-tuned Transformer (0.9388 ± 0.0080). On HUFLIT-Career (119k lines, 6 templates, grouped by client IP), Isolation Forest achieves the highest F1-score of 0.6292 ± 0.0235, outperforming PCA (0.6065 ± 0.1426) and deep models (DeepLog and TCN both obtaining 0.5916 ± 0.0096 under NLL thresholding). Under the tested configuration, TCN demonstrates a notable classification advantage on BGL and competitive inference latency compared to LSTM and Transformer models, while PCA remains competitive on short-sequence datasets such as HDFS. The contributions include: (i) a reproducible evaluation pipeline that prevents data leakage; (ii) a cross-dataset evaluation between HDFS, BGL, and HUFLIT-Career; (iii) a grouping strategy study on fixed and sliding windows; (iv) ablation studies on Drain3 threshold, top-k, and mask ratio; and (v) inference-latency and qualitative explainability analyses.

⋆ Corresponding authors: H. M. Tran (hatm@huflit.edu.vn) and T. H. Trinh (tht.csec2005@gmail.com, 23dh113972@st.huflit.edu.vn). The complete source code and raw training logs to ensure experiment reproducibility are publicly available at: https://github.com/thtcsec/log-anomaly-tcn-transformer.

Keywords: Log anomaly detection · event-transition network · Transformer · Drain3 · AIOps · distributed systems.

1 Introduction

Modern large-scale computing infrastructures, including distributed storage systems, supercomputers, web services, and cloud platforms, generate large volumes of logs with diverse formats and frequencies. These system logs capture both the operational state and the behavioral patterns of running services, while manually crafted detection rules quickly become obsolete as the infrastructure evolves. Effective log anomaly detection therefore requires analysis at the event sequence level rather than individual log lines. To capture these complex sequential relationships, we model the log stream as a directed event-transition network, where the vertices represent parsed event templates and the directed edges represent sequential transitions between events. A sequence of log messages generated by a specific operational unit (such as a Hadoop block or a client IP session) is thus represented as a walk (trajectory) on this directed transition network. In web-based environments like HUFLIT-Career, this state-transition network is coupled with a bipartite user-service interaction network where client IPs act as users executing walks across system routes. Detecting system anomalies is then formulated as identifying anomalous walks or trajectory deviations on these state-transition and user-interaction networks. Unsupervised models learn the normal traversal probabilities across network nodes, flagging walks that visit rare transitions or display quantitative anomalies.

This paper is positioned as an applied research and system evaluation study, rather than proposing a new anomaly detection algorithm. The central idea is to treat the event template sequence after Drain3 parsing as an intermediate representation layer: compact enough to run lightweight baselines, structured enough to extend to sequence models, and clear enough to map to operational units such as sessions, anonymized users, or time windows. Three research questions are addressed:

- **RQ1:** How does the method of constructing event sequences after Drain3 affect the anomaly detection signal?
- **RQ2:** How do the five scoring branches (reconstruction, isolation, next-event prediction, causal convolution, and masked event modeling) compare on the same event sequence?
- **RQ3:** Does the performance ranking of these branches remain consistent when moving from public benchmark logs to an institutional web-log environment?

The main contributions of this work are three-fold: (1) a unified leakage-aware log anomaly detection pipeline that structures raw logs into event walks on directed state-transition and interaction networks; (2) a cross-dataset evaluation across public benchmarks and a proprietary institutional web-log dataset, validating generalization via a strict IP-disjoint ablation; (3) a deployment trade-off

analysis comparing classification performance (F1-score), CPU/GPU inference latency, and qualitative explainability.

To justify the academic and practical merits of this work, we outline its positioning across four key pillars:

- **Necessary:** Modern systems generate gigabytes of log data per second. Traditional manual rules fail to capture contextual shifts. Unsupervised, sequence-level evaluation is necessary to automate anomaly detection in live systems.
- **Novelty:** This work provides a deployment-oriented, leakage-aware evaluation of frequency-based, isolation-based, causal sequence, and masked-event modeling branches under a unified Drain3-based pipeline across public benchmark logs and an institutional web-log dataset.
- **Significance:** The results show that while sequence-based deep models excel on complex logs (BGL), lightweight baselines like PCA and Isolation Forest are superior on structured logs (HDFS, HUFLIT-Career). TCN represents a highly significant alternative, achieving comparable performance to LSTM but with significantly lower CPU latency.
- **Applicability:** The pipeline is lightweight, uses Drain3 for online parsing, and operates on standard CPU/GPU resources, making it highly applicable to real-world security monitoring in enterprise environments.

Rather than proposing another detector, this work contributes a network-structured evaluation framework that identifies when lightweight frequency models, sequential predictors, and masked-event models are preferable under different log-network regimes.

2 Related Work

The earliest log anomaly detection methods typically represented log data as event count vectors or feature matrices, and then applied traditional machine learning models such as PCA, clustering, or Isolation Forest [10, 11]. PCA is used to separate normal behavior space and detect points deviating from the principal subspace. This group of methods is lightweight and easy to deploy, but usually ignores the temporal order and contextual relationships between log events. Recently, comprehensive empirical studies (e.g., [24]) have re-evaluated the trade-offs between classical machine learning and deep learning, indicating that traditional models remain highly competitive on specific structures and resource-constrained environments.

To address this limitation, sequential models like DeepLog [1] use LSTM networks to predict the next event based on previous events. LogAnomaly [13] and LogRobust [14] extend this direction by combining sequential and semantic log information to deal with unstable log data. Additionally, semi-supervised approaches such as PLELog [15] use probabilistic label estimation to reduce dependence on manual labels. Overall, RNN/LSTM architectures capture sequence relationships better than traditional statistical methods, but still suffer from sequential processing limitations and are difficult to parallelize.

To represent unstructured logs for machine learning models, log parsing techniques are used to map each log line to a template. Drain [3] and Drain3 are widely used due to their fixed-depth parse tree structure, which allows high-speed online parsing. Recently, LogPPT [7] used prompt-based few-shot learning to improve parsing accuracy when labeled data is scarce.

The Transformer architecture with its self-attention mechanism [4] has been widely applied in AIOps. LogBERT [2] employs a Transformer encoder combined with masked event modeling to learn bidirectional representations for normal log sequences, improving contextual anomaly detection. Survey papers also note a strong shift from traditional sequential models to self-supervised deep learning based on Transformers [16]. In parallel, Large Language Model (LLM) and prompt learning approaches such as LogPrompt [8] and LogGPT [9] enhance semantic understanding, but face major barriers in terms of inference cost and real-time latency.

More recently, research has moved towards leveraging advanced foundation models, active domain adaptation, and parameter-efficient fine-tuning (PEFT). For instance, LogLLM [20] integrates BERT for semantic feature extraction and Llama for sequence classification using a projecting layer, demonstrating robustness on unstable logs without parser dependencies. Under the tested context, PEFT approaches like LINEADAPTER [23] and language model fine-tuning frameworks like LogFiT [22] have been proposed to adapt large language representations to log anomaly detection and root cause analysis with minimal training cost. To maintain consistency across different system deployments, cross-system domain adaptation techniques (e.g., LogAction [25]) have been developed to transfer learned anomaly boundaries to target environments. Additionally, to balance the trade-off between execution speed and semantic reasoning, collaborative frameworks like CoLA [21] combine lightweight Small Detection Models (SDMs) as anomaly filters with Large Language Models (LLMs) serving as high-fidelity expert classifiers and explainers.

Table 1. Comparison of methodology groups and research gaps

Group	**Representatives**	**Key Limitation**	**Improvement in This Work**
Statistical	PCA, IF [10, 11, 24]	No temporal ordering	Lightweight baseline quantification
Sequential	DeepLog [1, 13]	Limited sequence length, sequential bottleneck	Parallel TCN integration
Transformer	LogBERT [2, 16]	Dependent on parsing quality	Decoupled parsing, grouping, and scoring
LLM / Foundation	LogGPT [9], LogPrompt [8], LogLLM [20], CoLA [21], LogFiT [22], LINEADAPTER [23], LogAction [25]	High inference latency and computational cost	We evaluate lightweight TCN and Transformer models for low-latency deployment

From existing works, four main research gaps can be identified: (i) traditional statistical methods do not learn the order of log events; (ii) sequential LSTM models are limited with long log sequences; (iii) Transformer models depend

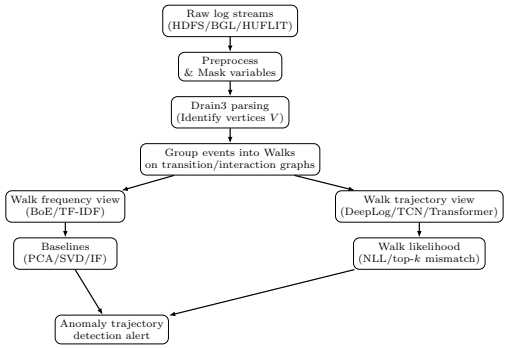

Fig. 1. Proposed log anomaly detection pipeline block diagram

heavily on the parsing step; (iv) mapping anomaly detection pipelines to large networks requires clarifying the grouping strategy and business scenarios.

To address these gaps, this paper builds a standardized pipeline that separates three phases (parsing, grouping, and scoring) using Drain3 as a preprocessing step to clean the token space. We evaluate in parallel the scoring branches including lightweight baselines (PCA, TruncatedSVD, Isolation Forest), sequential LSTM (DeepLog), the integrated Temporal Convolutional Network (TCN), and Transformer masked event modeling. Performance is carefully measured using Precision, Recall, and F1-score averaged over 5 different random seeds to ensure scientific reliability. To ensure experiment reproducibility and verification, the complete source code and raw training logs are publicly available at `https://github.com/thtcsec/log-anomaly-tcn-transformer`.

3 Proposed Methodology

3.1 Pipeline Overview

The proposed pipeline is designed to clearly separate three processing layers: log standardization, event sequence construction, and anomaly detection. The input is a raw log dataset $L = \{l_1, l_2, \ldots, l_N\}$ generated from the operating system. The output is an anomaly label or score $a(S)$ for each event sequence S. In HDFS experiments, a sequence corresponds to a Block ID; in BGL experiments, a sequence corresponds to a fixed-size time window. In broader deployment scenarios, a sequence can correspond to a login session, an anonymized user, a service, or a sliding time window.

The overall process consists of six steps: (1) **Log Collection**: receive logs from source systems; (2) **Preprocessing**: normalize variable fields such as timestamps and identifiers; (3) **Log Parsing**: use Drain3 to map each log line to an event template and event ID; (4) **Event Grouping**: group event IDs by Block ID or by session/time window to create event sequences; (5) **Feature Representation**: generate bag-of-events/TF-IDF representations for baselines or keep raw event sequences for deep models; (6) **Anomaly Detection**: compute anomaly scores using reconstruction error, Isolation Forest, or sequence learning models.

3.2 Preprocessing, Drain3 Parsing, and Event Grouping

The input is a raw log set $L = \{l_1, l_2, \ldots, l_N\}$. The process starts by normalizing variable parameters (IP, timestamp, ID) and parsing raw logs into event templates using Drain3 [3], which are then grouped into sequences.

Drain3 uses a fixed-depth tree to find the corresponding event template for each log line. The similarity between two log lines T_1, T_2 is calculated as the ratio of matching tokens:

$$Sim(T_1, T_2) = \frac{1}{n} \sum_{i=1}^{n} equ(t_{1,i}, t_{2,i}) \tag{1}$$

If $Sim(T_1, T_2) > st$ (similarity threshold, default 0.5), the log line is assigned to an existing template; otherwise, Drain3 creates a new template. After this step, each template is assigned an event ID, and the log sequence is represented as:

$$S = [E_1, E_2, ..., E_m] \tag{2}$$

For HDFS, the sequence S is grouped by Block ID. For BGL, it is grouped by non-overlapping windows of W=100 lines. We also support sliding window strategies to study the effect of the grouping strategy.

3.3 Lightweight Machine Learning Baselines

PCA and TruncatedSVD are applied to the bag-of-events vector representation $x = [count(E_j, S)]_{j=1}^{d}$ to learn the normal behavior subspace. The anomaly score is computed as the reconstruction error:

$$score(S) = \frac{1}{d} \sum_{j=1}^{d} (x_j - \hat{x}_j)^2 \tag{3}$$

where $\hat{x}$ is the reconstructed vector after projecting onto the low-dimensional space (the number of principal components is set to 20 by default). The anomaly threshold τ is chosen as the 95th percentile on the normal training set. A sequence S is labeled as anomalous if $score(S) > \tau$. Isolation Forest is applied to the TF-IDF feature representation of the event sequences. The model isolates sparse data points using random partitioning trees.

3.4 Deep Sequence Models and Transformer

DeepLog (LSTM): Replicated as next-event prediction using a 2-layer LSTM [1]. For each position i, the model predicts the probability of event E_i given the context of the previous g events:

$$\hat{P}(E_i \mid E_{i-g}, \ldots, E_{i-1}) = \text{softmax}(W_o h_i + b_o) \tag{4}$$

In top-k mode, a sequence is flagged as anomalous if at least one actual event lies outside the top K predictions with the highest probability. In NLL threshold mode, the anomaly score of the sequence is the average negative log-likelihood (NLL) of the events.

Temporal Convolutional Network (TCN): Integrated to improve parallelization and expand the receptive field. TCN utilizes dilated causal 1D convolutions [19]. The convolution operation on element s of the input sequence x with a filter f of size k and a dilation factor d is defined as:

$$F(s) = (x *_d f)(s) = \sum_{j=0}^{k-1} f(j) \cdot x_{s-d \cdot j} \tag{5}$$

A Chomp1d mechanism is used to crop the padding, ensuring causality. TCN uses the same dataset, scoring methods (top-k and NLL), and 95th percentile threshold selection as DeepLog to ensure a fair comparison.

Transformer (LogBERT-inspired): Employs a Transformer Encoder architecture combined with masked event modeling to learn bidirectional contexts [2]. A proportion of events (default 15%) are replaced by a `[MASK]` token, and the model learns to recover them using cross-entropy loss:

$$L_{MLM} = -\sum_{i \in M} \log P(k_i \mid S_{masked}) \tag{6}$$

where M is the set of masked positions. During inference, the anomaly score of sequence S is the average MLM loss over the masked positions. An anomaly decision is made by comparing this score with a threshold τ selected as the 95th percentile on the normal validation set.

4 Experimental Setup

4.1 Datasets

The study utilizes HDFS and BGL from the LogHub benchmark [6, 12], alongside HUFLIT-Career, a production-style institutional web-log dataset collected over a one-month period in June 2026 from a live university web application server. HDFS contains logs from a Hadoop system and is labeled by Block ID (anomalous if the block contains errors); we evaluate on the first 500,000 log lines. BGL contains logs from a supercomputer system, where anomaly labels are assigned using non-overlapping windows of W=100 lines. HUFLIT-Career consists of HTTP access logs from an online university career service. To protect user privacy, all sensitive client identification data, including client IP addresses and query payloads, were fully hashed and anonymized. Client IP addresses are utilized strictly for sequence grouping and are not included as features for any model. Scanner-probe labels in HUFLIT-Career were assigned based on rule-based security signatures matching known automated scanning User-Agents (e.g., vulnerability scanners, requests/curl scripts), path traversal attempts, and

anomalous HTTP status code distributions (such as concentrated 403 Forbidden and 404 Not Found responses). These heuristic labeling rules were verified and validated by HUFLIT network security administrators to ensure label alignment. While the raw HTTP log files cannot be publicly released due to institutional privacy policies, the parsed event template sequences, transition distributions, and complete source code of the pipeline are fully public. The statistics of the three datasets after Drain3 parsing are described in Table 2.

Table 2. Statistics of HDFS, BGL, and HUFLIT-Career datasets used in the experiments

Metric	HDFS	BGL	HUFLIT-Career
Raw log lines	500,000	500,000	119,516
Grouping unit	Block ID	Window (W=100)	Client IP (W=10)
Number of sequences/windows	36,305	5,000	25,049
Number of normal sequences	34,558	2,657	24,549
Number of anomalous sequences	1,747	2,343 (47%)	500 (2.0%)
Number of event templates (Drain3)	137	198	6
Drain3 similarity threshold θ	0.5	0.5	0.5
Drain3 tree depth	4	4	4

Execution Environment: The experiments were deployed on a computer configured with an Intel Core i7-13620H CPU, an RTX 4050 6GB GPU, 16GB RAM, running Ubuntu 22.04, Python 3.9, and PyTorch 2.0.

4.2 Leakage-Aware Splitting and Threshold Selection

To prevent data leakage, all unsupervised models (PCA, SVD, Isolation Forest, DeepLog, TCN, and Transformer) are trained strictly on normal sequences. For HDFS and BGL, the data split uses a 70/30 ratio for train/test sets (classical baselines) and a 60/20/20 ratio for train/validation/test sets (neural sequence models). The anomaly scoring threshold is computed as the 95th percentile of normal reconstruction errors or negative log-likelihood (NLL) on the validation set, ensuring that test labels are never accessed during threshold determination.

For the production-style HUFLIT-Career logs where sequence grouping is performed by client IP address, we adopt a sequence-level random split. Client IP addresses are used solely as keys for grouping log events into sequences and are completely excluded from model features. While some IP addresses might span both the training and testing sets through separate sequence windows (a limitation shared with sequence-level LogHub splits), using Event ID templates as the sole feature prevents direct leakage. To empirically validate the leakage-free generalization of our pipeline on HUFLIT-Career, we perform an additional ablation study using a strict IP-disjoint split (where 70% of client IPs are used for training and the remaining 30% are reserved exclusively for testing, ensuring zero client overlap). Under this zero-leakage protocol, the PCA baseline achieves a Precision of 0.5081, a Recall of 0.9127, and an F1-score of 0.6528. This confirms that the event transition patterns learned from normal traffic generalize robustly to entirely unseen client sessions.

Across all experiments, PCA and TruncatedSVD fix the number of principal components at 20. Isolation Forest is configured with 200 decision trees trained on TF-IDF features. For the mini Transformer, the model consists of 2 encoder layers (d_{model}=64, 4 heads, feed-forward dimension 128) trained using the AdamW algorithm [17] for 5 epochs with a mask ratio r=0.15. The DeepLog and TCN networks use an embedding size of 64, a hidden size of 64, optimized using Adam with a learning rate of 1×10^{-3}, and trained for 10 epochs. To ensure experiment reliability, all results are reported as the mean ± standard deviation over 5 random seeds (21, 42, 84, 123, 777).

Evaluation Metrics: We use Precision, Recall, and F1-score to evaluate performance:

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F1 = \frac{2 \cdot P \cdot R}{P + R} \tag{7}$$

In actual network operations, a low Precision causes many false alarms, while a low Recall leads to missed incidents. Therefore, F1-score is the primary metric for comparing performance.

5 Results and Discussion

5.1 Cross-Dataset Performance Comparison

The results in Table 3 reveal that no single model dominates across all settings — a finding that underscores the importance of evaluating the entire pipeline rather than any individual algorithm. On HDFS, the lightweight baselines achieve competitive results: PCA obtains an F1-score of 0.5711 ± 0.0944, outperforming deep learning models including DeepLog LSTM and TCN (both 0.4613 ± 0.0199). The mini Transformer obtains a low F1-score of 0.2950 ± 0.0693. We note that the Transformer evaluated here is a lightweight variant optimized for low latency; a larger, more deeply tuned Transformer model might yield different classification results but at the cost of higher latency. This observation arises because HDFS is a highly structured log dataset with a small vocabulary (137 templates) and very short sequences (average length of 13.77 events). In such settings, anomalies are often manifested as distributional shifts in event frequencies, which are effectively captured by PCA reconstruction error on bag-of-events features. Conversely, deep sequence models do not benefit from their temporal modeling capacity due to sequence sparsity and the absence of complex contextual transitions.

Visual Analysis and Threshold Sensitivity: In addition to F1-score, the baselines are analyzed through ROC and Precision-Recall (PR) curves in Fig. 2. The results show that PCA and TruncatedSVD maintain better separability than Isolation Forest across most threshold ranges. For HDFS (which is highly imbalanced), the PR curves indicate that PCA/SVD maintain high Precision at high Recall levels, whereas Isolation Forest declines rapidly. The corresponding AUC-ROC scores for PCA, TruncatedSVD, and Isolation Forest on HDFS (with seed 42) are 0.7722, 0.7536, and 0.7136, respectively, and the Average Precision (AP) scores are 0.5613, 0.5676, and 0.1645, respectively.

Table 3. Cross-dataset F1-score comparison (mean ± std over 5 seeds). Bold indicates best performance; underline indicates second best. Detailed Precision and Recall metrics are available in the public repository.

Method	HDFS (F1)	BGL (F1)	HUFLIT-Career (F1)
PCA	**0.5711 ± 0.0944**	0.9217 ± 0.0098	0.6065 ± 0.1426
TruncatedSVD	0.4540 ± 0.1366	0.9356 ± 0.0117	0.0659 ± 0.1474
Isolation Forest	0.1328 ± 0.0103	0.0404 ± 0.0070	**0.6292 ± 0.0235**
Transformer (unsup.)	0.2458 ± 0.0639	0.9242 ± 0.0085	N/A
Transformer (val-tuned)	0.2950 ± 0.0693	0.9388 ± 0.0080	0.3520 ± 0.0304
DeepLog (top-k)	0.4613 ± 0.0199	0.9820 ± 0.0042	0.0665 ± 0.0401
DeepLog (NLL)	0.3046 ± 0.0198	0.9381 ± 0.0075	0.5916 ± 0.0096
TCN (top-k)	0.4613 ± 0.0199	**0.9826 ± 0.0022**	0.0451 ± 0.0167
TCN (NLL)	0.3042 ± 0.0236	0.9474 ± 0.0035	0.5916 ± 0.0096

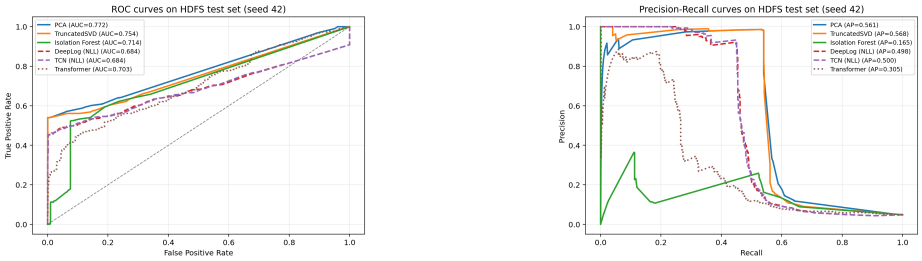

Fig. 2. ROC (left) and Precision-Recall (right) curves on HDFS.

In contrast, on BGL, the deep sequence models show strong results under the tested configuration: TCN (top-k) achieves the highest F1-score of 0.9826 ± 0.0022, followed closely by DeepLog (0.9820 ± 0.0042). In NLL mode, TCN (0.9474 ± 0.0035) continues to outperform DeepLog (0.9381 ± 0.0075) and the Transformer (0.9388±0.0080). Notably, the traditional baselines PCA and TruncatedSVD also perform well on BGL (0.9217 and 0.9356 F1-scores, respectively), indicating that BGL sequence structures are inherently separable.

On the institutional web log dataset, HUFLIT-Career, traditional machine learning models outperform deep sequence models. Isolation Forest achieves the highest F1-score of 0.6292 ± 0.0235, followed closely by PCA at 0.6065 ± 0.1426. Among sequence-based models, DeepLog (NLL) and TCN (NLL) obtain a competitive F1-score of 0.5916 ± 0.0096, while their top-k mismatch mode performs poorly (0.0665 ± 0.0401 and 0.0451 ± 0.0167). This divergence reveals an architectural insight: because HUFLIT-Career has a compact vocabulary of only 6 templates, event transitions remain highly regular, rendering next-event prediction mismatches ineffective. In contrast, NLL thresholding captures subtle likelihood shifts, yielding a robust 0.5916 F1-score. The high performance of Isolation Forest and PCA indicates that anomalies in simple-vocabulary logs are characterized more by frequency deviations than by sequential pattern shifts. Thus, scoring branch selection should be guided by vocabulary complexity and dataset characteristics rather than model capacity alone.

5.2 Grouping Strategy Study (RQ1)

To address RQ1, we evaluate the anomaly detection performance on BGL under four grouping configurations: non-overlapping fixed windows of size W=50, 100, 200, and a sliding window of size W=100 with stride 50. Table 4 shows the F1-score results for PCA, DeepLog (top-k), and TCN (top-k). Because this study focuses on the relative effect of grouping configurations, all results in Table 4 are reported under a single-seed controlled setting on a 200k BGL log subset (which has a different anomaly ratio) and should not be directly compared with the multi-seed aggregate results in Table 3.

Table 4 shows that window size W impacts PCA performance: F1-score is low at $W = 50$ and $W = 100$, but improves at $W = 200$ as larger windows stabilize frequency distributions. Conversely, TCN and DeepLog remain robust across all configurations. The sliding window (W=100, stride = 50) yields the best performance (DeepLog 0.9004, TCN 0.9032) by increasing training sample density and mitigating boundary effects.

Table 4. F1-score under different grouping configurations on BGL (seed 42)

Grouping Configuration	PCA	DeepLog (top-k)	TCN (top-k)
$W = 50$ (non-overlap)	0.1500	0.8556	0.8750
$W = 100$ (non-overlap)	0.1353	0.8571	0.8769
$W = 200$ (non-overlap)	0.4381	0.8810	0.8506
$W = 100$, stride 50 (sliding)	0.2314	0.9004	0.9032

5.3 Parameter Sensitivity, Latency, and Error Analysis

To understand the role of the three key hyperparameters, we perform three ablation studies on HDFS 200k (with seed 42): (i) Drain3 similarity threshold θ, (ii) DeepLog top-k parameter K, and (iii) Transformer mask ratio r. Table 5 and Table 6 summarize the results.

The ablation results provide three concrete insights. (A) The Drain3 similarity threshold has a strong impact on the pipeline quality: θ=0.3 leads to over-merging (only 25 templates, resulting in low F1 due to mixing different events); θ=0.7 causes template explosion (2,455 templates, causing F1 to collapse to 0.0446); θ=0.5 is the sweet spot, which is used as the default value throughout the paper. (B) DeepLog top-k saturates around K=5–9 with high precision and stable recall; K=1 is too strict (precision of 0.05), while K=15 begins to miss anomalies. (C) The mask ratio for the mini Transformer achieves the highest F1-score at the default value of r=0.15 — aligning with the classic BERT mask ratio [4]; increasing r to 0.20–0.30 reduces effectiveness as too many tokens are masked. The vocabulary growth analysis shows that the number of templates increases from 13 (5k lines) to 137 (500k lines) in a logarithmic pattern, reflecting the template generation characteristics of real system logs.

Expected Error Analysis: Four main types of errors are observed: (i) FP on rare sequences — normal but rare event sequences are incorrectly flagged

Table 5. Hyperparameter ablation study on HDFS 200k (seed 42)

Configuration	Templates	Precision	Recall	F1
(A) Drain3 θ, evaluated on PCA reconstruction				
$\theta = 0.3$	25	0.3695	0.5215	0.4325
$\theta = 0.5$ **(default)**	68	0.4070	0.5550	**0.4696**
$\theta = 0.7$	2455	0.1000	0.0287	0.0446
(B) DeepLog top-k				
$K = 1$	68	0.0466	0.6403	0.0869
$K = 3$	68	0.9091	0.2878	0.4372
$K = 5$	68	0.9524	0.2878	0.4420
$K = 9$ **(default)**	68	0.9756	0.2878	**0.4444**
$K = 15$	68	0.9750	0.2806	0.4358
(C) Transformer mask ratio r				
$r = 0.10$	68	0.1615	0.2230	0.1873
$r = 0.15$ **(default)**	68	0.1968	0.2662	**0.2263**
$r = 0.20$	68	0.1711	0.2302	0.1963
$r = 0.30$	68	0.1701	0.2374	0.1982

as anomalies; (ii) FP on long sequences — reconstruction error naturally increases with sequence length, requiring normalization; (iii) FN when anomalies use common templates — requiring sequence models (TCN/DeepLog); (iv) template collision — Drain3 merges different logs due to a loose similarity threshold, requiring fine-tuning of θ.

Inference Latency and Deployment Cost: A real-world log monitoring system must measure warning latency alongside the F1-score. To establish a benchmark, we measure the per-sample inference latency for each branch with a batch size of 1 on an RTX 4050 Laptop GPU (Drain3 and baselines measured on CPU; DeepLog/Transformer measured on both CPU and GPU) with 50 warm-up runs, averaged over 1,000 HDFS samples (Table 6).

Table 6. Per-sample inference latency, batch size 1, averaged over 1,000 HDFS samples (unit: ms)

Branch	Mean	p95	p99	Throughput
Drain3 parse 1 line	0.007	0.010	0.023	147,458 logs/s
PCA reconstruction	0.063	0.153	0.219	15,869 sequences/s
TruncatedSVD	0.176	0.281	0.381	5,689 sequences/s
Isolation Forest	4.944	9.015	11.396	202 sequences/s
DeepLog (CPU)	1.743	2.086	2.407	574 sequences/s
DeepLog (GPU)	0.489	0.798	0.962	2,046 sequences/s
TCN (CPU)	1.648	2.096	2.609	607 sequences/s
TCN (GPU)	0.455	0.780	0.950	2,198 sequences/s
Transformer (CPU)	2.395	2.966	3.267	418 sequences/s
Transformer (GPU)	2.916	4.323	4.889	343 sequences/s

Three observations from Table 6: (1) Drain3 processes ∼147k logs/sec, which is not a bottleneck; (2) PCA/SVD are the fastest (< 0.2 ms); IF is 100× slower due to traversing 200 trees; (3) TCN and DeepLog benefit from GPU execution (latency drops from ∼1.7ms to 0.45ms), whereas the small Transformer does not benefit at batch size 1. Training times: Drain3 takes 3.49 s, PCA 0.74 s, and Transformer 10.75 s — a trade-off that needs consideration in real-time monitoring.

5.4 Applications and Limitations

The pipeline could potentially support detection of brute-force password scanning (repeated login failures), anomalous API utilization (new templates from Drain3), or service-overload patterns (HTTP 5xx sequences) in production networks after validation on corresponding production logs. This is a proposed application analysis that requires confirmation on anonymized production data.

5.5 Qualitative Explainability Analysis

To enhance operational utility, we present a qualitative explainability layer based on next-event prediction mismatches. Table 7 illustrates two anomaly cases from HDFS and BGL using TCN. When flagged, the system outputs the failed event position, the actual versus top-3 predicted templates, and a severity score (NLL). For instance, in BGL sequence `W20200`, the unexpected occurrence of `E18` (data TLB error) instead of normal transitions (`E3`, `E6`, `E4`) produces a high score (12.68), directing operators to a virtual memory fault.

Table 7. Qualitative explainability case studies for log anomalies

Log Set	Seq ID	Pos	Mismatch Event (Actual)	Top-3 Predicted	NLL	Interpretation
HDFS	blk_-68076...	4	E19: Receiving empty packet...	E5, E3, E4	12.39	block write failure
BGL	W20200	1	E18: data TLB error interrupt...	E3, E6, E4	12.68	kernel page fault

Threats to Validity: (i) experiments on public benchmarks are complemented by validation on an institutional web-log dataset, but testing at scale in real-time production environments is still required; (ii) domain shift exists between lab datasets and real logs; (iii) benchmark/institutional labels may contain noise; (iv) Drain3 is sensitive to the threshold θ; (v) performance depends heavily on grouping strategies and threshold percentiles; (vi) the mini Transformer is not deeply optimized; (vii) anonymization and warning validation mechanisms are required in practice.

6 Conclusion

This paper has constructed and evaluated a reproducible log anomaly detection pipeline based on Drain3 and event sequence analysis, comparing lightweight baselines and sequence models (TCN, Transformer, LSTM) in parallel on HDFS, BGL, and HUFLIT-Career across 5 random seeds. A key finding is that no single scoring branch dominates across all settings: PCA remains competitive on HDFS; TCN (top-k) achieves high F1-score (0.9826 ± 0.0022) on BGL with low CPU latency; and Isolation Forest outperforms other models on HUFLIT-Career due to the template vocabulary simplicity. These results suggest that practitioners should select scoring branches based on dataset characteristics. For operational

deployment, an explainability layer could report, for each anomalous sequence, the event position where prediction fails, the actual versus top-k predicted templates, and the per-event anomaly score contribution, thereby reducing manual triage effort. Future directions include: (i) expanding validation to other large-scale enterprise log repositories; (ii) incorporating semantic embeddings for log templates; (iii) developing the proposed explainability mechanism into a fully automated alert diagnosis module.

References

1. M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," in *Proc. ACM CCS*, 2017.
2. H. Guo, S. Yuan, and X. Wu, "LogBERT: Log Anomaly Detection via BERT," in *Proc. IJCNN*, 2021.
3. P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An Online Log Parsing Approach with Fixed Depth Tree," in *Proc. IEEE ICWS*, 2017.
4. A. Vaswani et al., "Attention Is All You Need," in *Proc. NeurIPS*, 2017.
5. S. He, J. Zhu, P. He, and M. R. Lyu, "Experience Report: System Log Analysis for Anomaly Detection," in *Proc. IEEE ISSRE*, 2016.
6. J. Zhu et al., "Tools and Benchmarks for Automated Log Parsing," in *Proc. ICSE*, 2019.
7. V. H. Le and H. Zhang, "Log Parsing with Prompt-based Few-shot Learning," in *Proc. ICSE*, 2023.
8. T. Zhang, X. Huang, W. Zhao, S. Bian, and P. Du, "LogPrompt: A Log-based Anomaly Detection Framework Using Prompts," in *Proc. IJCNN*, 2023.
9. X. Han, S. Yuan, and M. Trabelsi, "LogGPT: Log Anomaly Detection via GPT," in *Proc. IEEE BigData*, 2023.
10. V. Chandola, A. Banerjee, and V. Kumar, "Anomaly Detection: A Survey," *ACM Comput. Surv.*, vol. 41, no. 3, 2009.
11. F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. IEEE ICDM*, 2008.
12. J. Zhu, S. He, J. Liu, M. R. Lyu, and P. He, "Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics," in *Proc. IEEE ISSRE*, pp. 355–366, 2023.
13. W. Meng et al., "LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs," in *Proc. IJCAI*, pp. 4739–4745, 2019.
14. X. Zhang et al., "Robust Log-Based Anomaly Detection on Unstable Log Data," in *Proc. ESEC/FSE*, 2019.
15. L. Yang et al., "Semi-Supervised Log-Based Anomaly Detection via Probabilistic Label Estimation," in *Proc. ICSE*, 2021.
16. M. Landauer, S. Onder, F. Skopik, and M. Wurzenberger, "Deep Learning for Anomaly Detection in Log Data: A Survey," *Mach. Learn. Appl.*, vol. 12, 2023.
17. I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *Proc. ICLR*, 2019.
18. A. Oliner and J. Stearley, "What Supercomputers Say: A Study of Five System Logs," in *Proc. IEEE/IFIP DSN*, 2007.
19. S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," *arXiv:1803.01271*, 2018.
20. W. Guan, J. Cao, S. Qian, J. Gao, and C. Ouyang, "LogLLM: Log-Based Anomaly Detection Using Large Language Models," *arXiv preprint arXiv:2411.08561*, 2024. https://doi.org/10.48550/arXiv.2411.08561
21. X. Zhu et al., "CoLA: Model Collaboration for Log-based Anomaly Detection," *Proc. VLDB Endow.*, vol. 18, no. 11, pp. 3979–3987, 2025. https://doi.org/10.14778/3749646.3749668
22. C. Almodovar, F. Sabrina, S. Karimi, and S. Azad, "LogFiT: Log Anomaly Detection Using Fine-Tuned Language Models," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 2, pp. 1715–1723, 2024. https://doi.org/10.1109/TNSM.2024.3358730
23. X. Liu, W. Bao, Y. Zhang, Y. Li, R. Ranjan, and D. N. Jha, "LINEADAPTER: Parameter-Efficient Fine-Tuning for Log Anomaly Detection and Root Cause Analysis," *preprint*, 2025.
24. B. Yu et al., "Deep Learning or Classical Machine Learning? An Empirical Study on Log-Based Anomaly Detection," in *Proc. IEEE/ACM ICSE*, pp. 1–13, 2024. https://doi.org/10.1145/3597503.3623308
25. C. Duan et al., "LogAction: Consistent Cross-system Anomaly Detection through Logs via Active Domain Adaptation," *arXiv preprint arXiv:2510.03288*, 2025.